NetSec 3D: Final Technical Report

Project: Intelligent network security with IDS, IPS, Zero Trust, SIEM and EDR on a simulated smart campus, shown in 3D. **Branch:** `feature/campus-c1-design` (nothing pushed, nothing deployed).

> **Read this first.** All traffic in this project is **synthetic and deterministic**. No real packet is sent, captured or blocked, the endpoint CLI is not a real shell, and every number below comes from the simulation engine or from the test runs listed in section 18. Status labels used throughout: **Implemented and tested**, **Implemented, partly tested**, **Not done / planned**.

---

1. Executive summary

NetSec 3D is a teaching and demonstration application. A Python backend runs a deterministic discrete-event simulation of a campus network (30 devices plus an abstract Internet segment, 32 links, 4 VLANs, 7 IP subnets) and returns a time-stamped trace. A React and Three.js front end replays that trace as a 3D campus with moving packets, device panels, a SIEM console and an endpoint CLI. The backend decides what happens to every packet; the front end only draws it.

The project demonstrates five security capabilities (IDS, IPS, Zero Trust, SIEM, EDR) through nine presentation scenarios (P1 to P9) plus the original seven scenarios (S1 to S7). The last full verification passed: 1176 backend tests, 93 of 93 scenario checks across 12 runs, 349 frontend tests, a production build, 70 of 70 browser interaction checks and 35 of 35 browser demonstration checks.

Known weakness: on the integrated Intel UHD GPU used for testing, High quality runs at 27 to 35 fps (below the 30 fps target in the idle overview); Low quality runs at 44 to 55 fps (section 19).

2. Objectives

1. Show how a firewall, ACLs, IDS, IPS, Zero Trust, SIEM and EDR cooperate, and which component makes each decision.
2. Keep IDS passive and IPS inline, and never blur the two.
3. Make every outcome reproducible, so a presenter can repeat any scenario and get the same result.
4. Present it as a recognisable 3D campus that stays responsive on ordinary laptops.

3. Scope and what is out of scope

In scope: the simulation, the nine presentation scenarios, the 3D campus, the panels, the SIEM console, the EDR isolate and restore actions, the endpoint CLI and a read-only device CLI. Out of scope (and not claimed): real traffic, real attack tooling, machine learning detection, a database, user accounts, real device configuration, and any deployment of the work.

4. System architecture

- **Backend:** Python, FastAPI, a deterministic discrete-event engine (`backend/netsec/sim/`), security modules (`backend/netsec/security/`), CLI (`backend/netsec/cli/`), API routes (`backend/netsec/api/`).

- **Front end:** React 19, Vite, React Three Fiber, drei, postprocessing, Zustand. A trace player applies events at the playhead.
- **Contract:** `contracts/events.json` (version 1.1.0) defines the trace events; `contracts/topology_original.json` pins the original topology.
- **Authority:** the backend produces the whole trace. Operator actions (isolate, restore) are control events at the playhead, after which the engine re-simulates; everything before the playhead stays identical.
- **Production mode:** read-only. State-changing actions return HTTP 403 and runs do not write server state. Status: **Implemented and tested.**

5. Network topology

- 31 nodes (30 devices and the abstract `INET` segment), 32 links, 4 VLANs (10 Staff, 20 Admin, 30 Servers, 99 Security), 7 IP subnets.
- The original 17 nodes and 18 links, their ids and addresses are unchanged (a test pins them to `contracts/topology_original.json`). The extension adds 14 devices and 14 links: more staff and admin PCs, extra servers, and the security components (SIEM-1, EDR-MGMT-1, ZT-POLICY-1).
- Path of a staff request to the application server: PC, ACC-SW2, CORE-SW1 (ACL and Zero Trust enforcement point), IPS-1 (transparent, inline), SRV-SW3, APP-SRV. Status: **Implemented and tested.** See `docs/NETWORK_TOPOLOGY.md`.

6. IDS (passive detection)

IDS-1 receives SPAN copies of traffic entering CORE-SW1 (not VLAN 99). It raises alerts and syslog only. It never forwards, drops, blocks or rate-limits, and it never recolours the original packet. Detectors: D1 port scan, D2 brute force, D3 flood, D4 restricted service, D5 forbidden zone pair, D6 invalid TCP flags, with thresholds in the shipped rules. A test asserts that IDS-1 never emits a drop, block or IPS action event. Status: **Implemented and tested.**

7. IPS (inline prevention)

IPS-1 is a transparent bridge in VLAN 30. Order for each packet: block table, detectors on every arriving packet, policy for the current mode, enforcement (drop, reject, block source, rate-limit), forward. **Monitor** records "would block" and drops nothing; **Enforce** applies the action. Mode is chosen per run. Status: **Implemented and tested** (scenarios S2, S4 to S7 exercise both modes).

8. Firewall and ACL (static policy)

FW1 (stateful, with connection tracking) and the CORE-SW1 ACL `STAFF-IN` enforce static policy regardless of IPS mode. For example STAFF-IN rule 10 denies staff to DB-SRV tcp/3306, rule 20 denies staff to the Security VLAN, rule 100 permits the rest; FW1 OUTSIDE-IN rule 30 publishes HTTP to APP-SRV and everything else is denied by the implicit rule. Status: **Implemented and tested.**

9. Zero Trust

ZT-POLICY-1 is asked once per flow by the policy enforcement point CORE-SW1, after the ACL. A decision uses endpoint trust, role and the requested resource, and records its reasons. Example (P4): the admin role is explicitly permitted tcp/443 on SIEM-1 and tcp/8443 on EDR-MGMT-1; being an administrator alone grants nothing. Status: **Implemented and tested.**

10. SIEM

SIEM-1 receives events from the components that produced them (IDS, IPS, firewall, ACL, Zero Trust, EDR), groups related events into incidents (incident ids are scoped per run) and keeps an incident history. The front end shows a SIEM console on the Threats page and security events in the player. Status: **Implemented and tested** (backend tests and browser demonstration check).

11. EDR with isolate and restore

Each endpoint has a state (healthy, compromised, isolated). EDR-MGMT-1 receives agent reports over an out-of-band channel. Isolation drops all of that host's traffic in both directions (drop reason `edr_isolated`); restore removes the isolation. The operator can do this from the endpoint panel (Security tab) when the server allows it; scenarios P7 and P8 script the same actions. Status: **Implemented and tested.**

12. Presentation scenarios P1 to P9

Every scenario has a name, objective, source, destination, protocol and port, a Run button, a Reset button, an animation, events, an expected outcome, an explanation and SIEM events, and can be repeated with the same result. The table lists the design values from `backend/config/scenarios/P*.json`; the outcomes are asserted by `backend/tests/scenarios/test_presentation_scenarios.py` and checked again in the browser (section 18).

#	Name	Source to destination	Protocol / port	Duration	Expected outcome
P1	Normal communication	PC-1 to APP-SRV	TCP 443	18 s	Delivered; no alert, no IPS action, no SIEM event
P2	Unauthorized Security VLAN access	PC-1 to SIEM-1	ICMP	6 s	Dropped at CORE-SW1 by STAFF-IN:20; no reply; SIEM gets an enforcement event
P3	Database access restriction	PC-1 to DB-SRV	TCP 3306	3 s	SYN dropped at CORE-SW1 by STAFF-IN:10; never reaches IPS-1 or DB-SRV
P4	Authorized administrator access	ADMIN-PC to SIEM-1	TCP 443	13 s	Two Zero Trust allows for ADMIN-PC; PC-1's attempt is stopped by STAFF-IN:20
P5	External HTTP access	EXT-USR to APP-SRV	TCP 80	25 s	Crosses R1-EDGE and FW1 (OUTSIDE-IN:30), reply allowed by connection tracking, delivered
P6	External database access restriction	EXT-ATK to DB-SRV	TCP 3306	3 s	Dropped at FW1 (implicit deny); never enters the campus; IDS-1 cannot see it
P7	Compromised endpoint and EDR isolation	PC-2 to APP-SRV	TCP	32 s	EDR alert at 1 s; D1 scan detected by IDS-1 and IPS-1; after isolation PC-2's packets are dropped with `edr_isolated`; other endpoints keep working
P8	Endpoint recovery	PC-2 to APP-SRV	TCP 443	26 s	Pings dropped while isolated; restore at 5 s; PC-2 healthy; HTTPS works again
P9	SIEM event monitoring	PC-3 to APP-SRV	TCP 443	7 s	IDS-1 and IPS-1 each raise D6; the SIEM lists the events as one incident

Status: **Implemented and tested.**

13. Original scenarios S1 to S7

The original seven scenarios are unchanged. `test_extension_equivalence.py` shows that their events are identical to the golden traces apart from the run id and configuration hash, because the new security features are opt-in per scenario. The shipped-rules check passes 12 of 12 runs and 93 of 93 checks (S1 to S7, Monitor and Enforce).

14. Endpoint CLI

An interactive terminal in the endpoint panel supports `help`, `ipconfig`, `ping`, `tracert`, `telnet`, `curl` and `clear`. Commands are whitelisted and validated by regular expressions; nothing is executed on any machine. Each network command runs the real simulation engine, so its output (replies, time-outs, the hop where it stopped) comes from the backend's decisions, not from a script. Status: **Implemented and tested** (`backend/tests/security/test_campus_security.py`, browser check).

15. Device CLI (read-only)

`show` commands for R1-EDGE, CORE-SW1 and FW1 produce output from the topology, the rules and the hit counters. They cannot change anything. Status: **Implemented and tested** (`backend/tests/security/test_device_cli.py`).

16. 3D campus

- Seven zones, each with a recognisable pavilion, raised platforms, roads, labelled signs, four racks, glass walkways, a reception pavilion, an entrance arch and guard booth, networking cabinets, server cabinets and a SOC monitoring wall with stations. There is no grass, tree or imported model; the vehicles that were drawn in car parks were removed on request, leaving empty marked bays.
- Devices are drawn 1.5 times their original model size so that they stay distinguishable from the default camera. The overview camera frames the campus so that it fills most of the viewport.
- Decoration is pure data plus merged geometry; it adds no node, link or cable (a test pins this).
- Picking: devices first, then cables, then zones. A browser check clicks every one of the 31 devices, every one of the 32 cables and all 7 zones. Status: **Implemented and tested.**

17. Front end features

Scenario launcher, endpoint panel (Overview, CLI, Security tabs), device panel, packet panel, playback bar with event markers, camera presets, High/Low quality with automatic fall-back, SIEM console, Monitoring and Threats pages, and a banner that says the traffic is synthetic. Not done: a final UI polish pass (Stage 7 in `docs/FINAL_EXECUTION_PROGRESS.md`).

18. Testing and validation

Last full run on the final build:

Check	Result
Backend tests (pytest)	1176 passed
Scenario checks S1 to S7 (shipped rules)	12 of 12 runs, 93 of 93 checks
Frontend tests (Vitest)	28 files, 349 tests passed
Production build	succeeded

Check	Result
Browser interaction check (`scripts/check_interactions.mjs`)	70 of 70
Browser demonstration check (`scripts/check_demo_scenarios.mjs`)	35 of 35
Split-origin check (`python scripts/tasks.py split-check`)	20 of 20 checks, and the 34-step demo rehearsal 34 of 34, with the API on another origin (local only; nothing was run on Render)
Offline recovery check	not re-run in this session

Shipped-rules trace hashes (from the verification log): S1 monitor 57cf6670ce8a, S1 enforce 0faa2b791f6e, S2 monitor 307190852d2a, S2 enforce 44dd9e4738d8, S3 monitor bf20f361e402, S3 enforce ac6bc5d49145, S4 monitor 3ad9ecd2b964, S4 enforce 3c3d85f3f55b, S5 monitor 85dc9ea334fa, S6 enforce e1b4b9c05cc5, S7 monitor fabf892d87b9, S7 enforce bc1b9cbdbeb2 (rules hash 79c0676ad9f6). The tests ran on one Windows machine; no other platform was tested in this session.

19. Performance

Measured with `scripts/measure_matrix.mjs` (results in `docs/perf/final-matrix.json`) on an Intel UHD Graphics laptop GPU through the app's built-in browser. The scene draws 276 to 287 calls and 76k to 78k triangles. No console errors were seen.

Quality	Case	Frame rate	Worst 1% frame
High	idle, overview	27.3 fps	100 ms
High	idle, staff close-up	31.4 fps	50.2 ms
High	scenario 7 playing	33.3 fps	50 ms
High	S4 flood peak	34.7 fps	33.8 ms
Low	idle, overview	54.4 fps	33.4 ms
Low	idle, staff close-up	54.9 fps	33.5 ms
Low	scenario 7 playing	55.3 fps	33.5 ms
Low	S4 flood peak	43.7 fps	49.9 ms

Assessment: the 30 fps target is met on Low and in three of four High cases, and **missed in the High idle overview (27.3 fps)** on this integrated GPU. The application lowers quality automatically when the frame rate stays low and tells the user. A discrete GPU was not available for measurement, so no claim is made about one.

20. Security design principles

- Backend authority: the front end never decides an outcome.
- IDS is passive; only IPS (and static FW and ACL policy, Zero Trust and EDR) enforce.
- Defence in depth: perimeter firewall, core ACL, Zero Trust, inline IPS and endpoint isolation each act at a different place.
- Production is read-only; the API validates input, limits request rates and whitelists CLI commands.

21. Determinism and reproducibility

The same scenario, seed and rules give the same trace hash. Mid-run changes leave everything before the playhead identical. This is what makes every scenario repeatable in front of an audience.

22. Limitations

Listed in `docs/LIMITATIONS.md`. Principally: simulated traffic only, simplified protocol behaviour, no machine learning, no persistence beyond a run, High-quality frame rate on integrated GPUs, and no multi-platform test in this session.

23. Risks and how they are handled

Risk	Handling
Low frame rate on the demo machine	Low quality and automatic fall-back; backup recording recommended
Operator actions on a shared server	Production is read-only; actions are available only in the local demo
Misreading synthetic results as real	The banner and every document say the traffic is synthetic

24. How to run

```
cd C:\dev\NetSec-3D
python scripts/tasks.py setup
python scripts/tasks.py demo        # builds and serves everything on one port
python scripts/tasks.py verify      # all automated checks
```

See `docs/DEVELOPMENT.md` and `docs/DEMO_SCRIPT.md`.

25. Future work (not done)

A final UI polish pass; High-quality optimisation for integrated GPUs; measuring on other machines; more device CLI commands; persisted run history.

26. Conclusion

The five security capabilities, the nine presentation scenarios, both CLIs and the expanded 3D campus are implemented and pass the automated checks listed above. The two open items that matter are the High-quality idle frame rate on an integrated GPU (27 fps) and the missing polish pass. Nothing here claims real network behaviour: it is a deterministic simulation built to explain how the defences work together.